LINUX ASSIGNMENT

Aditi Biju

26001100690027

Write an AWK program for each question. Ignore the header row where appropriate.

1. Print the source IP and event from every record.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {print $2, $3}' auth.log
192.168.1.10 Login_Success
192.168.1.15 Login_Failed
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
10.0.0.12 Malware_Alert
192.168.1.10 File_Access
```

2. Display the time, source IP, and attempts in aligned columns using printf.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 && ($4=="HIGH" || $4=="CRITICAL") {print $2, $3, $4}' auth.log
10.0.0.8 Port_Scan HIGH
192.168.1.25 Login_Failed HIGH
10.0.0.12 Malware_Alert CRITICAL
```

3. Store the source IP and event in variables, then print them in a sentence.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {ip=$2; event=$3; print "Source IP", ip, "has event", event}' auth.log
Source IP 192.168.1.10 has event Login_Success
Source IP 192.168.1.15 has event Login_Failed
Source IP 10.0.0.8 has event Port_Scan
Source IP 192.168.1.25 has event Login_Failed
Source IP 10.0.0.12 has event Malware_Alert
Source IP 192.168.1.10 has event File_Access
```

4. Calculate the processing time for each record if one attempt takes 2 seconds.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {time=$5*2; print $2, time}' auth.log
192.168.1.10 2
192.168.1.15 6
10.0.0.8 16
192.168.1.25 14
10.0.0.12 8
192.168.1.10 4
```

5. Calculate the total number of attempts in the file.

6. Calculate the average number of attempts per record.

7. Count the number of Login_Failed records.

8. Start with a review quota of 10 and decrease it by one for every HIGH or CRITICAL record. Print the remaining quota.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {total+=$5} END {print "Total attempts:", total}' auth.log
Total attempts: 25

┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {total+=$5; count++} END {print "Average:", total/count}' auth.log
Average: 4.16667

┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 && $3=="Login_Failed" {count++} END {print "Failed logins:", count}' auth.log
Failed logins: 2

┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {quota=10} NR>1 && ($4=="HIGH" || $4=="CRITICAL") {quota--} END {print "Quota:", quota}' auth.log
Quota: 7
```

9. Replace Port_Scan with Network_Scan in the event field and print every record in CSV format.

10. Add one to the attempts field of each Login_Failed record and print every record in CSV format.

11. Print the source IP and event when severity is HIGH and attempts are greater than 5.

12. Print the source IP and event when severity is HIGH or CRITICAL.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {OFS=","} NR>1 {if ($3=="Port_Scan") $3="Network_Scan"; print $1,$2,$3,$4,$5}' auth.log
09:10,192.168.1.10,Login_Success,LOW,1
09:15,192.168.1.15,Login_Failed,MEDIUM,3
09:20,10.0.0.8,Network_Scan,HIGH,8
09:25,192.168.1.25,Login_Failed,HIGH,7
09:30,10.0.0.12,Malware_Alert,CRITICAL,4
09:35,192.168.1.10,File_Access,LOW,2

┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {OFS=","} NR>1 {if ($3=="Login_Failed") $5++; print $1,$2,$3,$4,$5}' auth.log
09:10,192.168.1.10,Login_Success,LOW,1
09:15,192.168.1.15,Login_Failed,MEDIUM,4
09:20,10.0.0.8,Port_Scan,HIGH,8
09:25,192.168.1.25,Login_Failed,HIGH,8
09:30,10.0.0.12,Malware_Alert,CRITICAL,4
09:35,192.168.1.10,File_Access,LOW,2

┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 && $4=="HIGH" && $5>5 {print $2, $3}' auth.log
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed

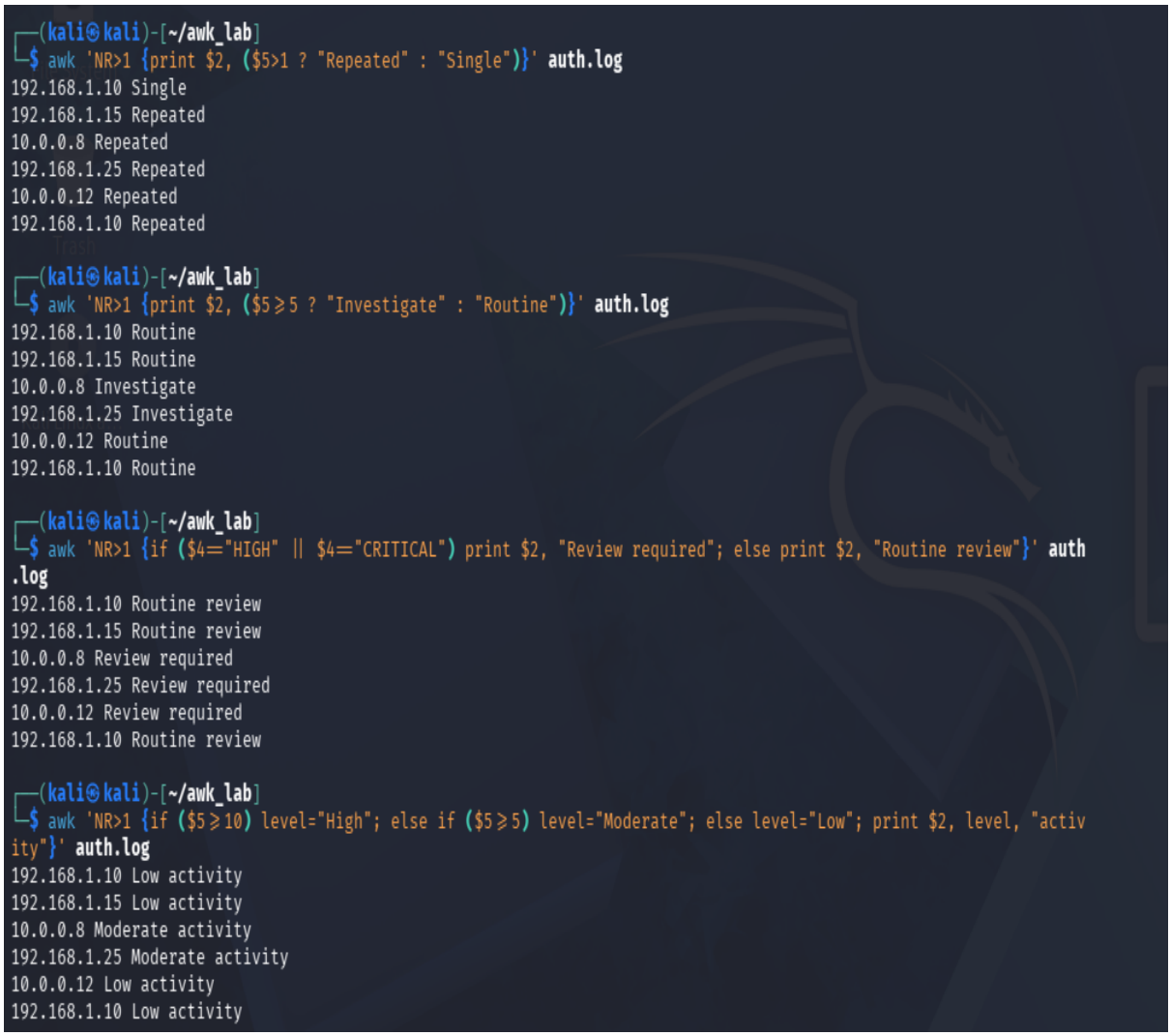
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 && ($4=="HIGH" || $4=="CRITICAL") {print $2, $3, $4}' auth.log
10.0.0.8 Port_Scan HIGH
192.168.1.25 Login_Failed HIGH
10.0.0.12 Malware_Alert CRITICAL
```

13. Use the conditional operator to label each record Repeated if attempts exceed 1; otherwise label it Single.

14. Use the conditional operator to label each record Investigate if attempts are at least 5; otherwise label it Routine.

15. Use if–else to print Review required for HIGH or CRITICAL records and Routine review for all other records.

16. Use if–else if–else to classify attempts as High activity (10 or more), Moderate activity (5–9), or Low activity (fewer than 5).



17. Use a for loop to print Check 1 through Check 5.

18. Use a while loop to count down from 10 to 1.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {for (i=1; i<=5; i++) print "Check", i}' auth.log
Check 1
Check 2
Check 3
Check 4
Check 5

┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {i=10; while (i>=1) {print i; i--}}' auth.log
10
9
8
7
6
5
4
3
2
1
```

19. Use a do–while loop to count down from 10 to 1.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {i=10; do {print i; i--} while (i>=1)}' auth.log
10
9
8
7
6
5
4
3
2
1
```

20. Print each record's source IP, event, attempts, and activity classification in aligned columns. At the end, print the total records and total attempts.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {printf "%-18s %-15s %d\n", $2, $3, $5; count++; total+=$5} END {print "Records:", count; print "Total
 attempts:", total}' auth.log
192.168.1.10       Login_Success   1
192.168.1.15       Login_Failed    3
10.0.0.8           Port_Scan       8
192.168.1.25       Login_Failed    7
10.0.0.12          Malware_Alert   4
192.168.1.10       File_Access     2
Records: 6
Total attempts: 25
```

Login monitoring: A security analyst considers five or more failed attempts suspicious. Write an AWK script that prints the source IP and attempts for matching Login_Failed entries. At the end, print how many suspicious entries were found.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 && $3=="Login_Failed" && $5>=5 {print $2, $5}' auth.log
192.168.1.25 7
```

Incident summary: Write an AWK script that counts the entries in each severity category and prints a summary in an END block.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {count[$4]++} END {for (s in count) print s, count[s]}' auth.log
LOW 2
HIGH 2
CRITICAL 1
MEDIUM 1
```

Processing estimate: Assume each attempt requires 2 seconds of analyst review. Calculate the review time for each entry and the total review time for the file. Use printf to present the results neatly.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'NR>1 {time=$5*2; print $2, time " seconds"; total+=time} END {print "Total review time:", total, "seconds"}
' auth.log
192.168.1.10 2 seconds
192.168.1.15 6 seconds
10.0.0.8 16 seconds
192.168.1.25 14 seconds
10.0.0.12 8 seconds
192.168.1.10 4 seconds
Total review time: 50 seconds
```

Log normalization: A monitoring tool requires the event name Failed_Login instead of Login_Failed. Update the matching field and print all records as CSV. The source file should remain unchanged.

```
┌──(kali㉿kali)-[~/awk_lab]
└─$ awk 'BEGIN {OFS=","} NR>1 {if ($3="Login_Failed") $3="Failed_Login"; print $1,$2,$3,$4,$5}' auth.log
09:10,192.168.1.10,Login_Success,LOW,1
09:15,192.168.1.15,Failed_Login,MEDIUM,3
09:20,10.0.0.8,Port_Scan,HIGH,8
09:25,192.168.1.25,Failed_Login,HIGH,7
09:30,10.0.0.12,Malware_Alert,CRITICAL,4
09:35,192.168.1.10,File_Access,LOW,2
```

Priority queue: Assign each entry a priority: Immediate for CRITICAL, Soon for HIGH, and Routine for all others. Print the time, source IP, event, and assigned priority.

